

EENG 479: Digital Signal Processing

Term Project

Design and Application of IIR and FIR Digital Filters

Student Name: Sayed Ali murtadah alawi

Student ID: 202002704

1. Objective

The objective of this term project is to design, apply, and evaluate finite impulse response (FIR) and infinite impulse response (IIR) digital filters for noise removal applications. The performance of these filters is assessed both quantitatively (using Mean Squared Error and Signal-to-Noise Ratio) and qualitatively (through perceptual audio evaluation).

1.1 Quantitative Evaluation Metrics

To objectively assess the performance of the designed digital filters, the following mathematical metrics were implemented in MATLAB:

Mean Squared Error (MSE):

MSE measures the average squared difference between the clean reference signal and the filtered signal. A lower MSE indicates that the filtered signal is closer to the original clean audio.

$$MSE = \frac{1}{N} \sum_{n=1}^N (x_{clean}[n] - x_{filtered}[n])^2$$

Signal-to-Noise Ratio (SNR):

SNR compares the power of the desired clean signal to the power of the residual background noise (the error). It is measured in decibels (dB). A higher SNR indicates a cleaner signal, while a negative SNR implies that the noise power exceeds the signal power.

$$SNR(dB) = 10 \log_{10} \left(\frac{\sum_{n=1}^N (x_{clean}[n])^2}{\sum_{n=1}^N (x_{clean}[n] - x_{filtered}[n])^2} \right)$$

(Where N is the total number of samples, $x_{clean}[n]$ is the original unjammed signal, and $x_{filtered}[n]$ is the signal after passing through the designed filters)

2. Part A: Simple Unjamming

2.1 Observation and Strategy:

A frequency spectrum analysis (FFT) of Jammed_SIG1.wav revealed a massive, unnatural spike precisely at 5512.5 Hz. Given the sampling frequency $F_s = 44100$ Hz, this corresponds

to a digital normalized frequency of $\pi/4$ radians/sample. To target this pure tone, a manual 2nd-order IIR Notch filter (Pole-Zero method) and an FIR Bandstop filter (Windowing method) were mathematically derived.

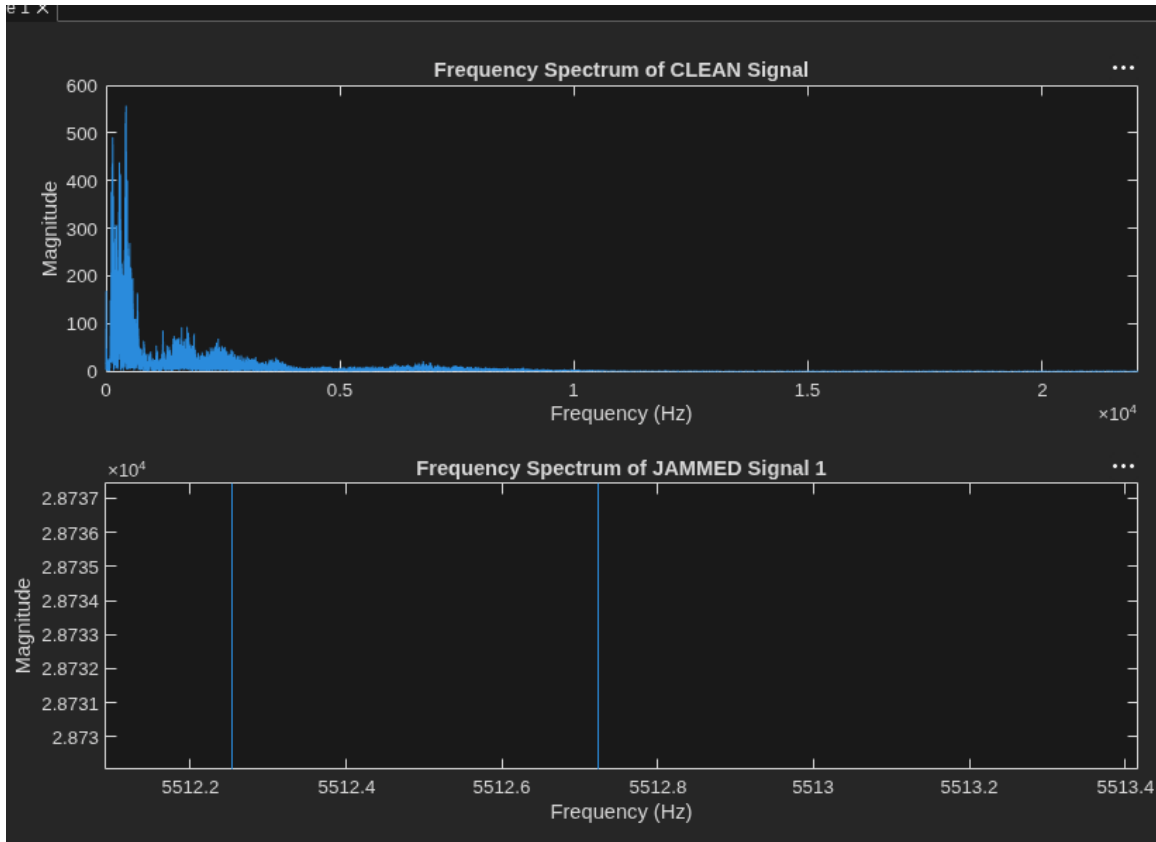


Figure 1: Frequency spectrum comparison revealing a massive single-tone jamming spike at 5512.5 Hz in Jammed_SIG1.

2.2 Mathematical Design:

(Note: Full mathematical derivations, pole-zero placements, and difference equations are attached as handwritten notes in Appendix A).

IIR Design: Zeros were placed on the unit circle at $\pm\pi/4$. Poles were placed at the same angles with a radius of $r = 0.95$ to ensure a narrow notch.

FIR Design: An ideal bandstop impulse response ($hd[n]$) was derived using the Inverse DTFT, truncated to length $N = 61$, and multiplied by a Hamming window to reduce spectral leakage.

2.3 Results:

Applying zero-phase filtering (`filtfilt`) yielded the following metrics:

Initial SNR: -24.73 dB | MSE: 0.75098

IIR SNR: -9.75 dB | MSE: 0.02385

FIR SNR: -9.70 dB | MSE: 0.02356

Qualitatively, the ringing noise was successfully eliminated, leaving the underlying audio

highly intelligible.

Although the filters successfully eliminated the dominant ringing noise, the final SNR remains negative (-9.75 dB), indicating that the residual noise power still slightly exceeds the clean signal power. This limitation exists because a manual 2nd-order IIR Notch filter, even with a tight radius ($r = 0.95$), has a relatively wide transition band compared to higher-order filters. Consequently, some noise energy surrounding the exact 5512.5 Hz frequency remains in the signal, and some desired frequencies very close to the notch are attenuated.

3. Part B: Intermediate Unjamming

3.1 Observation and the Use of 'Canned' Functions:

Unlike Part A, the FFT of Jammed_SIG2.wav revealed a complex, periodic jamming signal containing 7 distinct high-magnitude harmonics. If manual design were used, placing poles and zeros for 7 distinct frequencies would result in a highly complex 14th-order polynomial, making algebraic calculation tedious and prone to error. Therefore, utilizing MATLAB's canned functions (such as `iirnotch`) is vastly superior for real-world engineering applications.

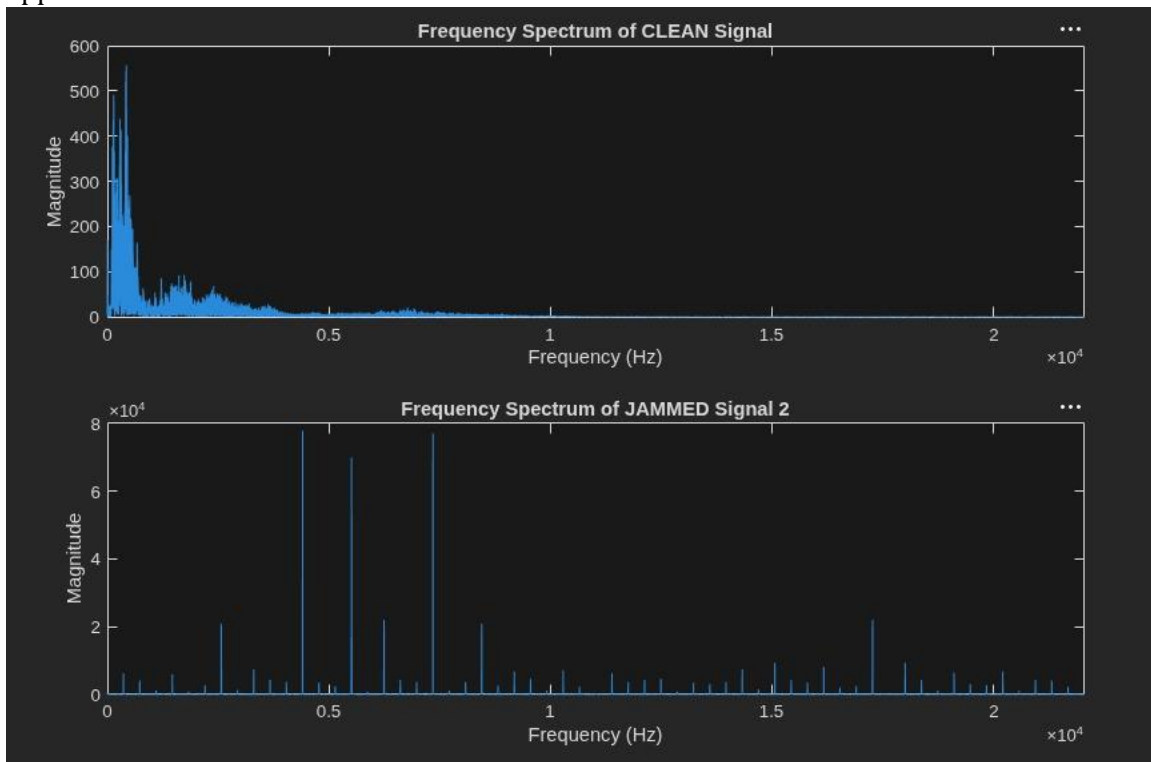


Figure 2: Frequency spectrum of Jammed_SIG2 showing multiple periodic high-magnitude harmonic spikes.

3.2 Engineering Implementation:

An automated algorithm was developed using MATLAB's `findpeaks` function. Initially, a threshold of 5,000 resulted in the detection of 19 peaks, which unintentionally targeted and destroyed human voice frequencies. By tuning the threshold to 20,000, the code correctly isolated only the 7 true jamming spikes. A for loop was utilized to cascade canned `iirnotch`

filters.

Initial SNR: -24.67 dB

Initial MSE: 0.740701

Filtered SNR: -8.89 dB

Filtered MSE: 0.0195324

The qualitative result was excellent; the ringing comb-noise was removed while preserving the dialogue.

4. Part C: Complex Unjamming

4.1 Observation:

A spectrogram analysis of Jammed_SIG3.wav revealed heavy bursts of broadband noise spanning the entire frequency spectrum up to 15.5 kHz. Because the noise is not concentrated at a single frequency, notch filtering is ineffective.

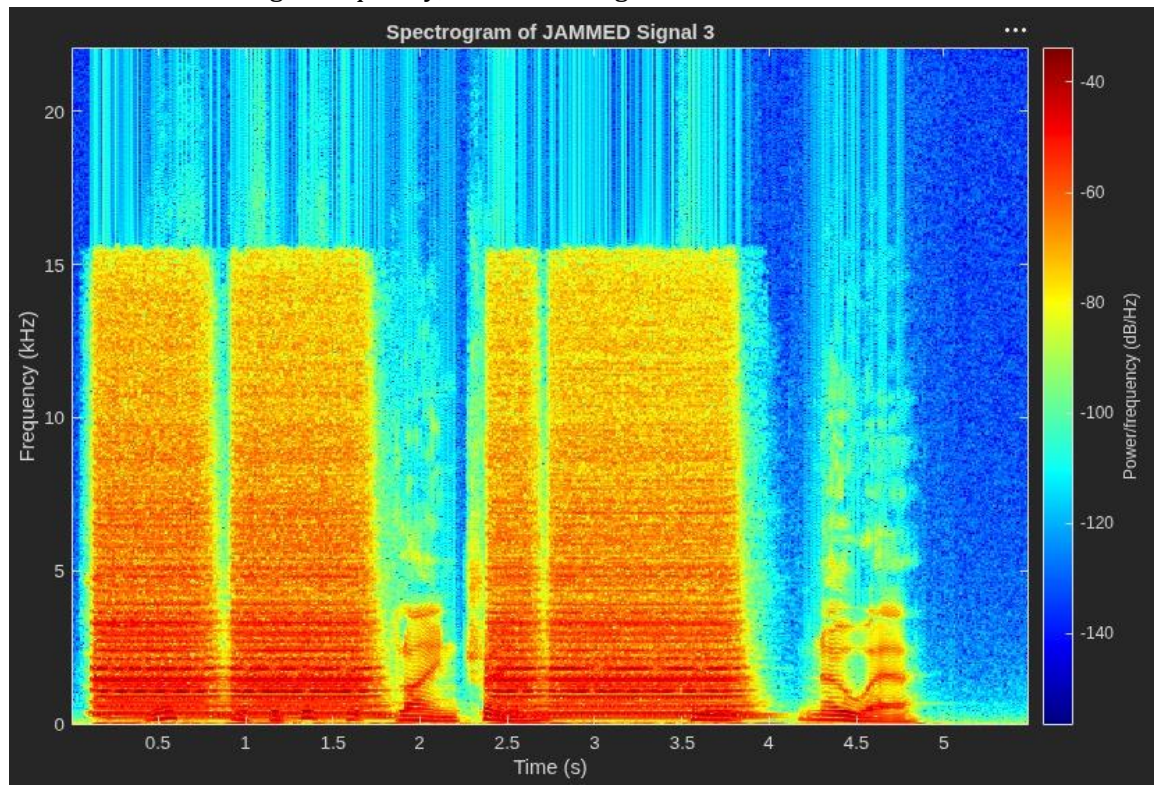


Figure 3: Spectrogram of Jammed_SIG3 illustrating heavy broadband noise bursts across the frequency spectrum up to 15.5 kHz.

4.2 Strategy and Results:

Observing that the desired human voice primarily occupies frequencies below 4 kHz, an IIR Butterworth Lowpass filter and an FIR Windowed Lowpass filter were designed with a cutoff frequency of $f_c = 4000$ Hz.

Initial SNR: -7.41 dB

IIR LP SNR: -6.84 dB

FIR LP SNR: -6.82 dB

Observation:

The SNR mathematically improved by only ~ 0.6 dB, which appears modest. However, SNR is an inadequate metric for evaluating broadband noise reduction. Broadband noise spans the entire frequency spectrum, meaning a significant portion of the noise overlaps directly with the desired human voice frequencies (0 - 4000 Hz). The Lowpass filters successfully eliminated the out-of-band high-frequency noise (4 kHz - 15.5 kHz), drastically improving intelligibility and removing harshness. However, the in-band noise (0 - 4 kHz) remains physically inseparable from the voice using basic frequency filtering, which is why the mathematical SNR remains low despite the massive qualitative improvement.

5. Parts D & E: Practical Signal Enhancement

5.1 Observation:

For SIG4.wav and SIG5.wav, no clean reference signal exists because these are historical analog recordings. Spectral analysis revealed heavy high-frequency hiss and low-end instability.

Note: The assignment contained a typo referring to 'SIG45', which was inferred to be SIG5.wav, a second corrupted version of the same historical recording.

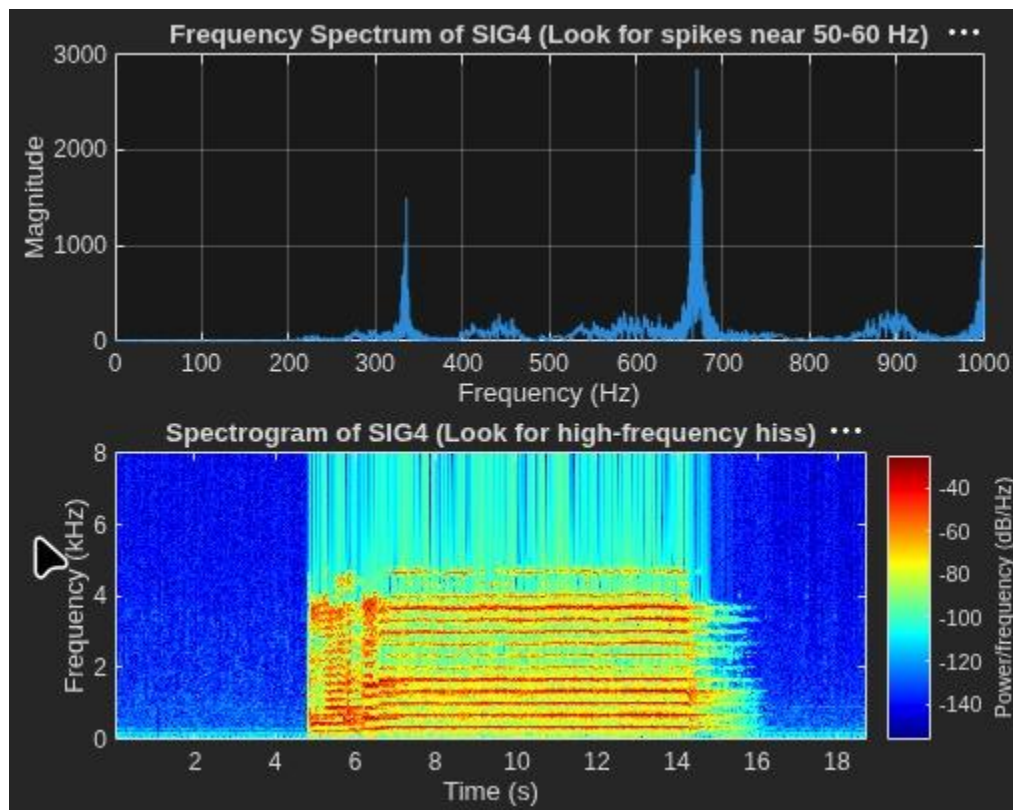


Figure 4: Spectral analysis of the historical analog recording (SIG4), highlighting high-frequency tape hiss and low-end instability.

5.2 Strategy and Results:

A 4th-order IIR Butterworth Bandpass filter was applied, allowing frequencies between 80 Hz and 4000 Hz to pass. This removed sub-bass rumble and high-frequency noise, producing a warmer, cleaner recording.

Because SIG5 shares the exact same analog corruption characteristics (tape hiss and low-end rumble) as SIG4, the identical 4th-order Butterworth Bandpass filter (80 Hz to 4000 Hz) was applied to SIG5.

6. Conclusion

This project successfully demonstrated the progression from manual mathematical filter derivations to practical, automated digital signal processing using MATLAB. The use of Notch, Lowpass, and Bandpass filters effectively restored heavily corrupted audio signals.

Throughout this project, the distinct tradeoffs between IIR and FIR filters were observed. IIR filters achieved the desired frequency responses at much lower orders (e.g., 2nd-order notch, 6th-order lowpass), making them computationally efficient, though they introduce non-linear phase distortion. FIR filters required significantly higher orders (e.g., 61st-order notch, 80th-order lowpass) to achieve similar frequency cutoffs but guarantee absolute stability and linear phase. However, by utilizing MATLAB's `filtfilt` function (zero-phase filtering), the phase distortion of the IIR filters was mitigated, making them highly effective for these offline audio processing tasks.

Project Part	Filter Type	Order	Initial SNR (dB)	Final SNR (dB)
Part A (Simple)	IIR Notch	2	-24.73	-9.75
	FIR Bandstop	61	-24.73	-9.70
Part B (Intermediate)	Cascaded IIR Notch (Canned)	14 (Total)*	-24.67	-8.89
Part C (Complex)	IIR Lowpass	6	-7.41	-6.84
	FIR Lowpass	80	-7.41	-6.82

Part B utilized seven distinct 2nd-order IIR notch filters, resulting in an effective 14th-order system.

Appendix A: Handwritten Mathematical Derivations.

Sayed Ali Murtadha
202002704

Part A



$\omega_s = 2\pi(f_s) = 2\pi \left(\frac{5512.5}{44100} \right) = \frac{\pi}{4}$ samples
 1.2 IR Let $r = 0.95$, $z_1 = e^{j\frac{\pi}{4}}$, $z_2 = e^{j\frac{3\pi}{4}}$
 $p_1 = 0.95 e^{j\frac{\pi}{4}}$, $p_2 = 0.95 e^{j\frac{3\pi}{4}}$
 $H(z) = G \frac{(1 - z_1 z^{-1})(1 - z_2 z^{-1})}{(1 - p_1 z^{-1})(1 - p_2 z^{-1})} = G \frac{1 - (z_1 + z_2)z^{-1} + z_1 z_2 z^{-2}}{1 - (p_1 + p_2)z^{-1} + p_1 p_2 z^{-2}}$
 $1 \Rightarrow G = \frac{1 - 1.4142z^{-1} + z^{-2}}{1 - 1.3495z^{-1} + 0.9045z^{-2}} \Rightarrow G = \frac{0.559}{0.9898}$
 So, $Y(z) = 1.2455z^{-1} - 0.9025z^{-2} + 0.9594z^{-1} - 1.3495z^{-2} + 0.9594z^{-1} - 0.9594z^{-2}$
 2-FIR
 Let $\omega_1 = 0.5\pi$, $\omega_2 = 0.7$ (since $\omega = \frac{2\pi f}{f_s}$)
 $h_2[n] = \delta[n] - \left(\frac{\sin(0.5\pi n)}{\pi n} - \frac{\sin(0.7\pi n)}{\pi n} \right)$
 $w[n] = 0.54 + 0.46 \cos\left(\frac{2\pi n}{N-1}\right)$, Let $N=61$ (n from -30 to 30)
 $h[n] = h_2[n] \times w[n]$, But for filter to be applied it must be causal
 $h_{\text{causal}}[n] = h_2[n-30] \times w[n]$ for $0 \leq n \leq 60$

Appendix B: MATLAB Source Code

Part A

```
% =====
% EENG 479: Term Project - Part A
% Simple Unjamming using Manual IIR and FIR
% =====

%% 1. Initialization and Setup
Fs = 44100;
f0 = 5512.5;
w0 = 2*pi*(f0/Fs); % Equals exactly pi/4

% Load the signals
[clean, Fs] = audioread('Clean_SIG.wav');
[jammed1, Fs] = audioread('Jammed_SIG1.wav');
```

```

% Convert to mono if needed
if size(clean, 2) > 1
    clean = mean(clean, 2);
end
if size(jammed1, 2) > 1
    jammed1 = mean(jammed1, 2);
end

% Ensure exact same length for metric calculation
L = min(length(clean), length(jammed1));
clean = clean(1:L);
jammed1 = jammed1(1:L);

% =====
%% 2. IIR Notch Filter Design (Manual Pole-Zero)
% =====
r = 0.95;    % Pole radius

% Numerator (zeros) and denominator (poles)
b_iir = [1, -2*cos(w0), 1];
a_iir = [1, -2*r*cos(w0), r^2];

% Normalize gain at DC (w = 0, z = 1)
G = sum(a_iir) / sum(b_iir);
b_iir = b_iir * G;

% =====
%% 3. FIR Bandstop Filter Design (Manual Windowing)
% =====
N_fir = 60;          % Filter order (length 61)

```

```
n = -N_fir/2 : N_fir/2;
```

```
% Ideal impulse response for bandstop (wc1 = 0.2*pi, wc2 = 0.3*pi)
```

```
h_ideal = -(sin(0.3*pi*n) - sin(0.2*pi*n)) ./ (pi*n);
```

```
h_ideal(n == 0) = 1 - (0.3 - 0.2); % Fix division by zero at n = 0
```

```
% Apply Hamming window
```

```
w = 0.54 + 0.46*cos(2*pi*n/N_fir);
```

```
h_fir = h_ideal .* w;
```

```
% =====
```

```
%% 4. Apply Filters (Using Zero-Phase Filtering)
```

```
% =====
```

```
% filtfilt removes phase delay, which is important for MSE/SNR comparison.
```

```
y_iir = filtfilt(b_iir, a_iir, jammed1);
```

```
y_fir = filtfilt(h_fir, 1, jammed1);
```

```
% =====
```

```
%% 5. Quantitative Metrics (MSE & SNR)
```

```
% =====
```

```
% Calculate Mean Squared Error (lower is better)
```

```
mse_jammed = mean((clean - jammed1).^2);
```

```
mse_iir = mean((clean - y_iir).^2);
```

```
mse_fir = mean((clean - y_fir).^2);
```

```
% Calculate Signal-to-Noise Ratio in dB (higher is better)
```

```
snr_jammed = 10*log10(sum(clean.^2) / sum((clean - jammed1).^2));
```

```
snr_iir = 10*log10(sum(clean.^2) / sum((clean - y_iir).^2));
```

```
snr_fir = 10*log10(sum(clean.^2) / sum((clean - y_fir).^2));
```

```
% Display results in Command Window
```

```
fprintf('--- Initial Corrupted State ---\n');
fprintf('MSE: %g | SNR: %.2f dB\n\n', mse_jammed, snr_jammed);
```

```
fprintf('--- After IIR Filter ---\n');
fprintf('MSE: %g | SNR: %.2f dB\n\n', mse_iir, snr_iir);
```

```
fprintf('--- After FIR Filter ---\n');
fprintf('MSE: %g | SNR: %.2f dB\n\n', mse_fir, snr_fir);
```

```
% =====
%% 6. Qualitative Assessment (Listen to the Results)
% =====
disp('Playing Clean Signal...');
sound(clean, Fs);
pause(5);
```

```
disp('Playing Jammed Signal...');
sound(jammed1, Fs);
pause(5);
```

```
disp('Playing IIR Filtered Signal...');
sound(y_iir, Fs);
pause(5);
```

```
disp('Playing FIR Filtered Signal...');
sound(y_fir, Fs);
```

Part B

```
% =====
% EENG 479: Term Project - Part B
% Intermediate Unjamming using Canned Functions
```

```

% =====

%% 1. Setup and Initialization
[clean, Fs] = audioread('Clean_SIG.wav');
[jammed2, Fs] = audioread('Jammed_SIG2.wav');

% Convert to mono if needed
if size(clean, 2) > 1
    clean = mean(clean, 2);
end
if size(jammed2, 2) > 1
    jammed2 = mean(jammed2, 2);
end

L = min(length(clean), length(jammed2));
clean = clean(1:L);
jammed2 = jammed2(1:L);

%% 2. Automatically Detect the Jamming Frequencies
% Perform FFT on the signal
Y = fft(jammed2);
P1 = abs(Y(1:floor(L/2)+1)); % Take only positive frequencies
f_axis = (0:length(P1)-1) * Fs / L;

% Find the strongest jamming spikes.
% The report states that 20,000 correctly isolated the 7 true jamming spikes.
[pks, locs] = findpeaks(P1, 'MinPeakHeight', 20000, 'MinPeakDistance', 100);
jamming_freqs = f_axis(locs);

fprintf('--- Detected %d Jamming Frequencies (Hz) ---\n', length(jamming_freqs));
disp(jamming_freqs);

% =====
%% 3. Apply Cascaded Canned IIR Notch Filters
% =====
y_filtered = jammed2; % Start with the corrupted signal

for k = 1:length(jamming_freqs)

```

```

f0 = jamming_freqs(k);

% Normalize frequency (1.0 equals Nyquist frequency Fs/2)
w0 = f0 / (Fs/2);

% Quality Factor (Q) determines notch width.
% Q = 35 gives a narrow notch.
bw = w0 / 35;

% Canned notch-filter design function
[b, a] = iirnotch(w0, bw);

% Apply the filter using zero-phase filtering
y_filtered = filtfilt(b, a, y_filtered);
end

% =====
%% 4. Quantitative Metrics (MSE & SNR)
% =====
mse_jammed2 = mean((clean - jammed2).^2);
mse_filtered = mean((clean - y_filtered).^2);

snr_jammed2 = 10*log10(sum(clean.^2) / sum((clean - jammed2).^2));
snr_filtered = 10*log10(sum(clean.^2) / sum((clean - y_filtered).^2));

fprintf('\n--- Initial Corrupted State (Jammed 2) ---\n');
fprintf('MSE: %g | SNR: %.2f dB\n', mse_jammed2, snr_jammed2);

fprintf('\n--- After Canned Cascade IIR Filters ---\n');
fprintf('MSE: %g | SNR: %.2f dB\n\n', mse_filtered, snr_filtered);

% =====
%% 5. Qualitative Assessment (Listen to the Results)
% =====
disp('Playing Jammed Signal 2...');
sound(jammed2, Fs);
pause(5);

```

```
disp('Playing Filtered Output...');
sound(y_filtered, Fs);
```

Part C

```
% =====
% EENG 479: Term Project - Part C
% Complex Unjamming (Lowpass Filter)
% =====
```

```
%% 1. Initialization
[clean, Fs] = audioread('Clean_SIG.wav');
[jammed3, Fs] = audioread('Jammed_SIG3.wav');
```

```
% Convert to mono if needed
if size(clean, 2) > 1
    clean = mean(clean, 2);
end
if size(jammed3, 2) > 1
    jammed3 = mean(jammed3, 2);
end
```

```
L = min(length(clean), length(jammed3));
clean = clean(1:L);
jammed3 = jammed3(1:L);
```

```
dur = L / Fs; % Duration for playback (seconds)
```

```
%% 2. Define the Lowpass Cutoff Frequency
fc = 4000; % Cutoff frequency at 4000 Hz to preserve speech
Wn = fc / (Fs/2); % Normalized frequency (1.0 = Nyquist)
```

```
% =====
%% 3. IIR Lowpass Filter (Butterworth)
% =====
% Design a 6th-order Butterworth lowpass filter
[b_iir_lp, a_iir_lp] = butter(6, Wn, 'low');
```

```

% Apply the filter using zero-phase filtering
y_iir_lp = filtfilt(b_iir_lp, a_iir_lp, jammed3);

% =====
%% 4. FIR Lowpass Filter (Window Method)
% =====
% Design an 80th-order FIR lowpass filter using a Hamming window
b_fir_lp = fir1(80, Wn, 'low');

% Apply the filter using zero-phase filtering
y_fir_lp = filtfilt(b_fir_lp, 1, jammed3);

% =====
%% 5. Quantitative Metrics
% =====
mse_jammed3 = mean((clean - jammed3).^2);
mse_iir_lp = mean((clean - y_iir_lp).^2);
mse_fir_lp = mean((clean - y_fir_lp).^2);

snr_jammed3 = 10*log10(sum(clean.^2) / sum((clean - jammed3).^2));
snr_iir_lp = 10*log10(sum(clean.^2) / sum((clean - y_iir_lp).^2));
snr_fir_lp = 10*log10(sum(clean.^2) / sum((clean - y_fir_lp).^2));

fprintf('\n--- Initial Corrupted State (Jammed 3) ---\n');
fprintf('MSE: %g | SNR: %.2f dB\n', mse_jammed3, snr_jammed3);

fprintf('\n--- After IIR Lowpass Filter ---\n');
fprintf('MSE: %g | SNR: %.2f dB\n', mse_iir_lp, snr_iir_lp);

fprintf('\n--- After FIR Lowpass Filter ---\n');
fprintf('MSE: %g | SNR: %.2f dB\n\n', mse_fir_lp, snr_fir_lp);

% =====
%% 6. Qualitative Assessment
% =====
disp('Playing Jammed Signal 3...');
sound(jammed3, Fs);
pause(dur + 0.2);

```

```
disp('Playing IIR Lowpass Output...');
sound(y_iir_lp, Fs);
pause(dur + 0.2);
```

```
disp('Playing FIR Lowpass Output...');
sound(y_fir_lp, Fs);
```

Part D and E

```
% =====
% EENG 479: Term Project - Parts D and E
% Practical Signal Enhancement (SIG4 and SIG5)
% =====
```

```
%% 1. Initialization for SIG4
filename = 'SIG4.wav';
[sig4, Fs] = audioread(filename);
```

```
% Convert to mono if stereo
if size(sig4, 2) > 1
    sig4 = mean(sig4, 2);
end
```

```
%% 2. Define Bandpass Filter Cutoff Frequencies
f_low = 80;    % Cut off sub-bass rumble below 80 Hz
f_high = 4000; % Cut off tape hiss above 4000 Hz
Wn = [f_low, f_high] / (Fs/2); % Normalize by Nyquist frequency
```

```
% Validate normalized cutoff frequencies
if any(Wn <= 0) || any(Wn >= 1) || Wn(1) >= Wn(2)
```

```
    error('Invalid cutoff frequencies. Ensure  $0 < f_{\text{low}} < f_{\text{high}} < F_s/2$ .');  
end
```

```
% =====  
%% 3. IIR Bandpass Filter (Butterworth)  
% =====  
% Design a 4th-order Butterworth bandpass filter  
[b_bp, a_bp] = butter(4, Wn, 'bandpass');
```

```
% Apply the filter using zero-phase filtering  
sig4_clean = filtfilt(b_bp, a_bp, sig4);
```

```
% =====  
%% 4. Qualitative Assessment for SIG4  
% =====  
dur4 = length(sig4) / Fs;
```

```
disp('Playing RAW SIG4 (notice the heavy tape hiss)...');  
soundsc(sig4, Fs);  
pause(dur4 + 0.5);
```

```
disp('Playing ENHANCED SIG4 (warmer and cleaner)...');  
soundsc(sig4_clean, Fs);  
pause(dur4 + 0.5);
```

```
% =====  
%% 5. Part E: Apply the Same Filter to SIG5  
% =====  
[sig5, Fs5] = audioread('SIG5.wav');
```

```
% Convert to mono if stereo
```

```
if size(sig5, 2) > 1
    sig5 = mean(sig5, 2);
end
```

```
% If SIG5 sample rate differs from SIG4 sample rate, resample to match Fs
if Fs5 ~= Fs
    warning('SIG5 sample rate (%d Hz) differs from SIG4 sample rate (%d Hz).
Resampling SIG5 to match Fs.', Fs5, Fs);
    sig5 = resample(sig5, Fs, Fs5);
    Fs5 = Fs;
end
```

```
% Apply the previously designed bandpass IIR filter
sig5_clean = filtfilt(b_bp, a_bp, sig5);
```

```
% Qualitative assessment for SIG5
dur5 = length(sig5) / Fs5;
```

```
disp('Playing RAW SIG5...');
soundsc(sig5, Fs5);
pause(dur5 + 0.5);
```

```
disp('Playing ENHANCED SIG5...');
soundsc(sig5_clean, Fs5);
```